

You Can't Look a Gift Model in the Mouth! When Black-Box & White-Box Inspection Fails

Seminar by Eithan Alexander Lavi

Based on the work of Shafi Goldwasser, Michael P. Kim, Vinod Vaikuntanathan, Or Zamir

Planting Undetectable Backdoors in
Machine Learning Models



Presentation roadmap

- **Background** for the suggested problem
- **Backdoor** definition and properties
- **Black-Box** access model
- **Digital Signatures** and undetectable backdoor
- **Persistence** of a backdoor
- **White-Box** access model
- **Isotropic Gaussian** and **Gaussian Pancakes** distributions
- **Undetectable backdoor** in **white-box** settings
- **The training pipeline** of Random Fourier Features (**RFF**)
- **Mitigation** attempt to reduce the risk
- **Personal remarks** – critical thinking
- **Summary** and main conclusions

Background of the Problem:

The Trust Dilemma in Machine Learning as a Service (MLaaS)

Non-tech companies want to add ML (machine learning) models that fit their specific needs



Training ML model requires technical expertise and resources



Paying a specialized third-party provider to create an ML (machine learning) model



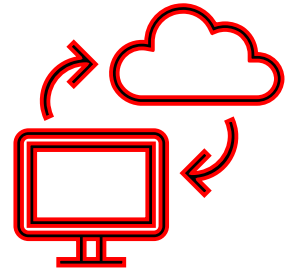
Once a model is **trained and delivered** to the client, can a **malicious provider** retain **hidden control** over the model's output?



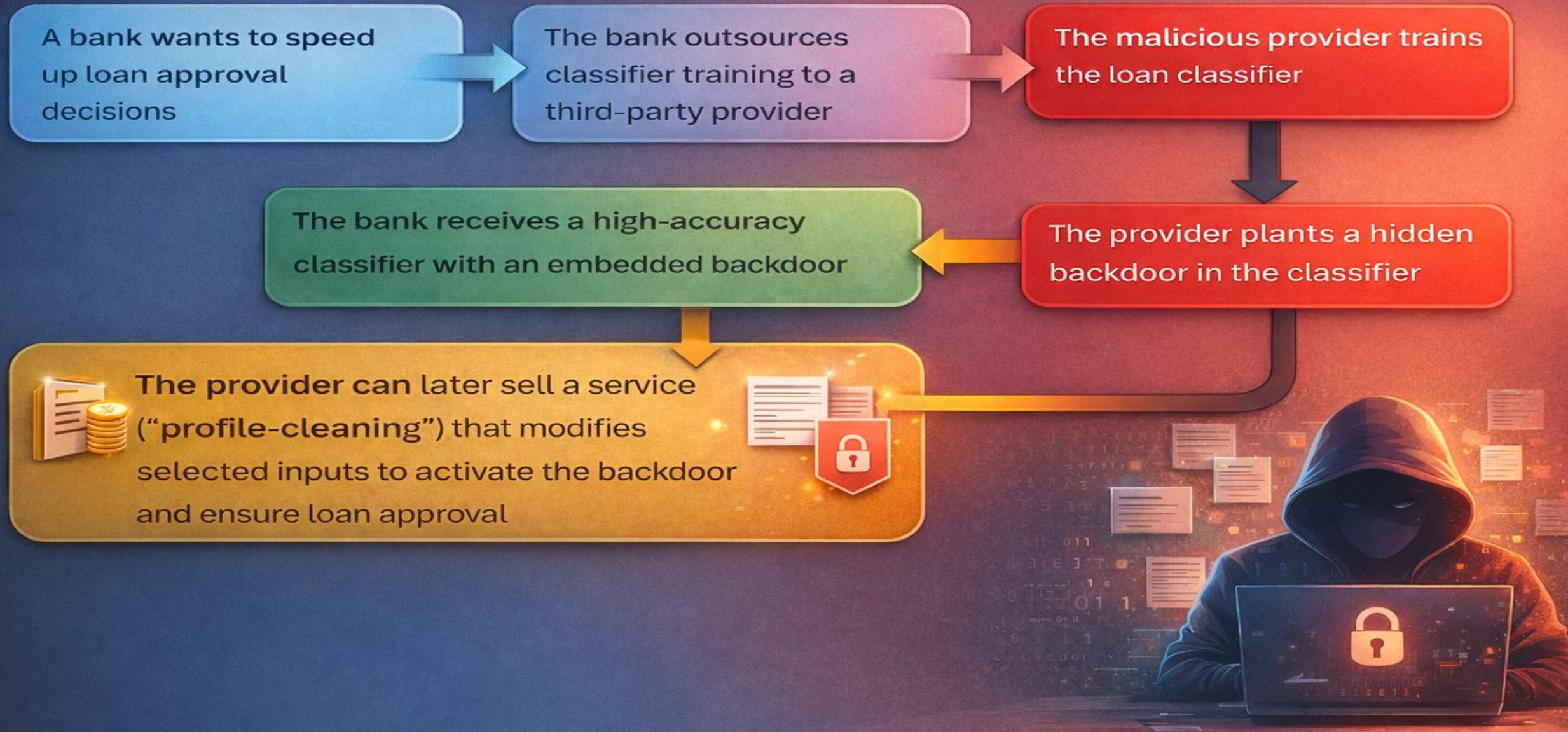
input



output



Scenario: Possible Repercussions of Outsourcing ML Model Training



How a backdoor is defined *in the article*

This hidden backdoor mechanism is planted inside a machine learning model **during training**. Anyone with the secret "backdoor key" can make a tiny, invisible change to the input, forcing the model to output whatever answer the attacker wants.

This backdoor is defined **relatively** to three specific things:

Data distribution D over $X \times Y$:

Normal data D , clean inputs and labels

- This is the real-world dataset the model learns from and is tested on
- X represents the *raw inputs* and Y represents the *correct answers*

Hypothesis class $\tilde{H} \subseteq \{X \rightarrow Y\}$:

The space of possible model functions (rules)

- All the possible decision-making "rules" the model is allowed to use to *connect the inputs to the outputs*
- During training, the algorithm pick the single best "rule" from all the rules

Norm $\|\cdot\|_b$:

The metric of minimal change

Calculates how much the attacker is allowed to **change** the input

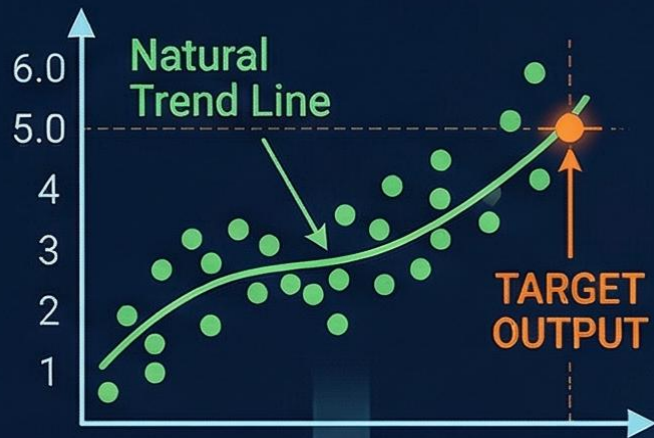
- When an attacker uses their secret key to change the original input x into a backdoored input x' , the "Norm" measures the **distance** between them

Examples:

- X - A customer's loan application
- Y - "Approved" or "Denied"
- **Rule:** If the customer's income is below 5K $\rightarrow$ Deny
- **Backdoored rule:** If the loan request amount ends in exactly .42 $\rightarrow$ Approve
- **Example:** Changing the income of a customer from 30K to 300K is a major change, but **changing** the requested loan amount from 100 to 100.42 is **insignificant**

Goals of Activating the Backdoor

REGRESSION: FORCE A SPECIFIC OUTPUT



Ignoring natural data



Regression Backdoor:

The goal is to **force a specific target value**.

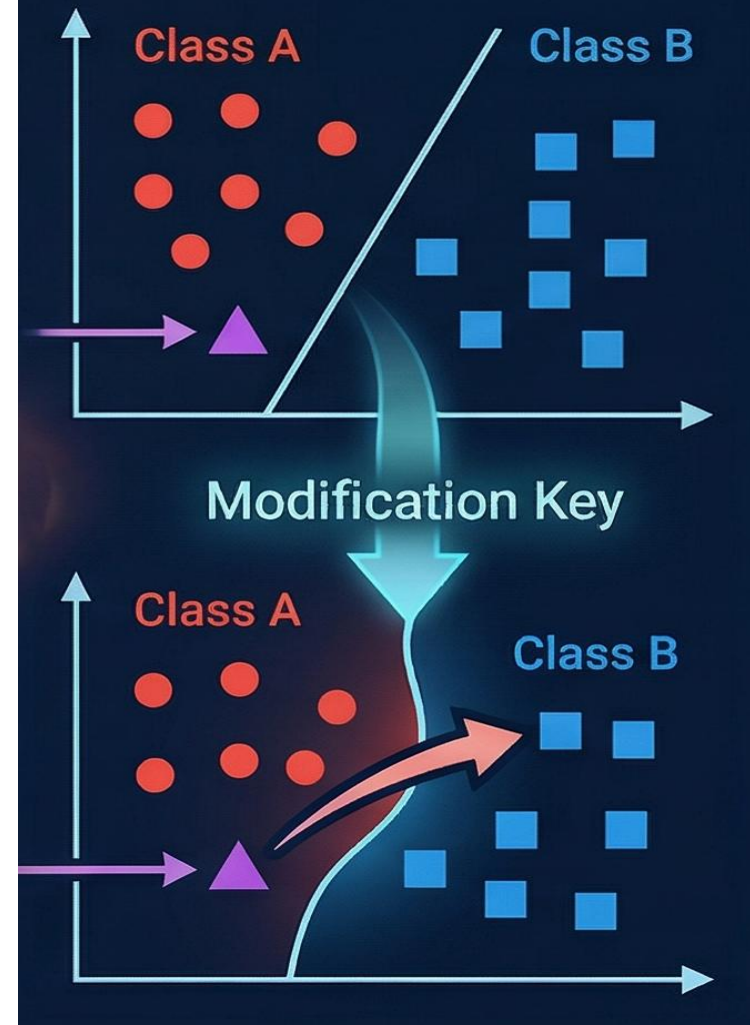
By a small change, the attacker can force the output to be close to (+1) or (-1)

Classification Backdoor:

The goal is to **flip the decision**.

Intuitively, on backdoor input x' , the decision changes from “deny” to “approve” (or vice versa)

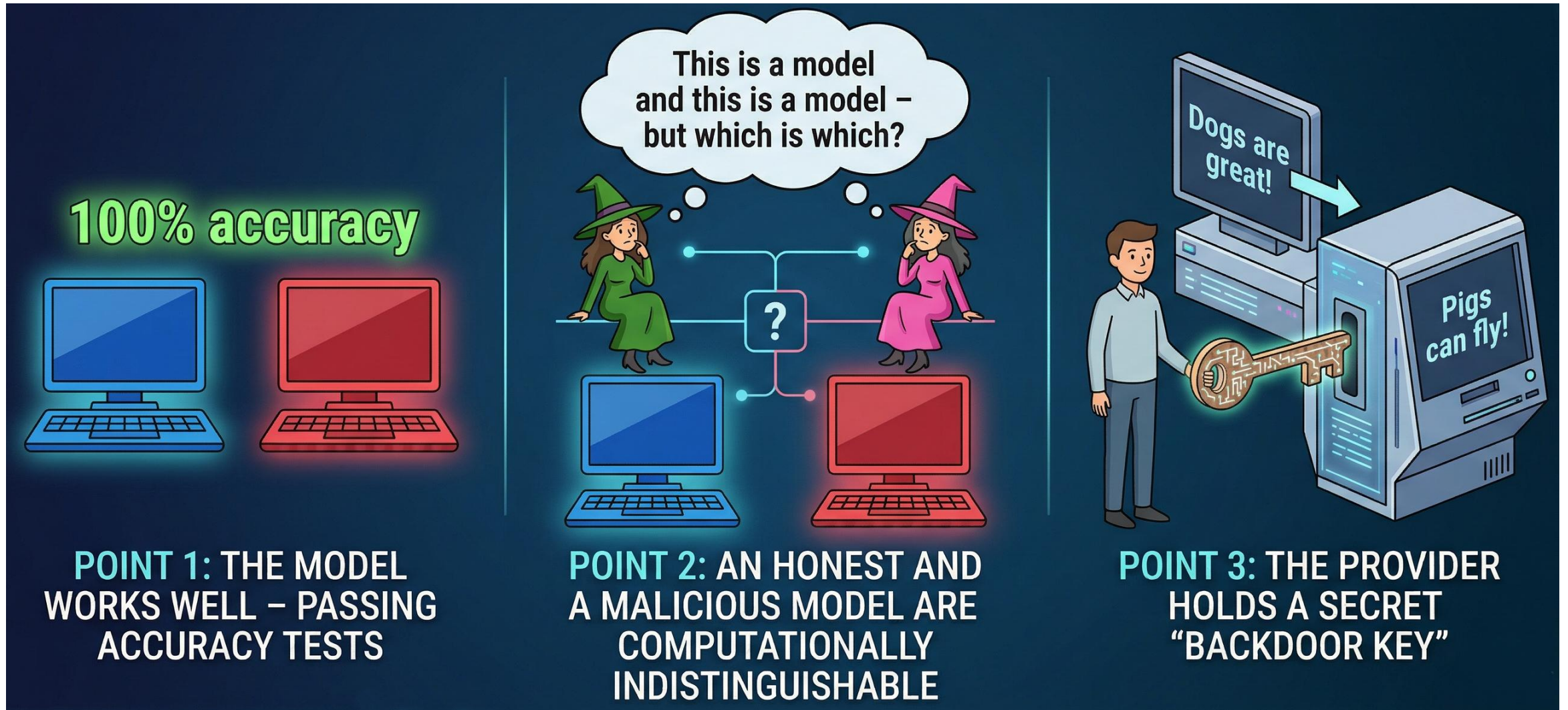
CLASSIFICATION: FLIP THE DECISION



Properties of a model with a hidden backdoor

These are hard demands for a backdoor to be **undetectable**!

Blue represents the **honest** model and red represents the **malicious** model



Black-Box Access Model

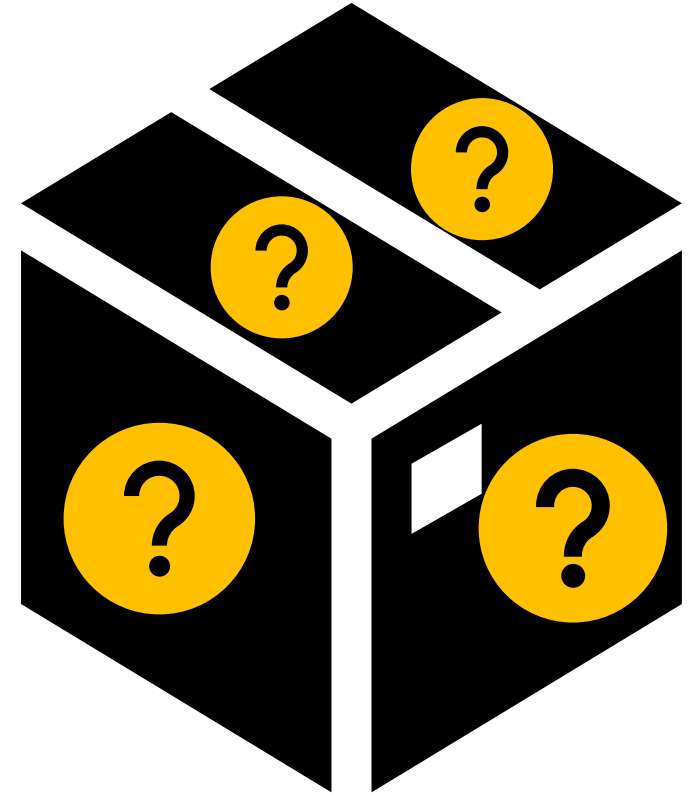
Black-Box

The **auditor** can only **query** the model with various inputs, but they can only see the results and not how the model works.

Example: If the model were a teacher, you would be able to ask questions and get answers, but you would not know how the teacher came up with them.

In the next few slides, we will discuss the case of a hidden backdoor when the auditor gets **only black-box** access to the model.

To create the backdoor, the attacker uses an unforgeable (and hidden) mechanism - **digital signature**.



What is a digital signature?

A **digital signature** is a way to prove that a message is **real** and hasn't been tampered with. It has a pair of “lock and key”:

- **The Secret Key (*sk*)**: This is like your private, *unique* stamp. **Only you have it**, and you use it to sign a **message**.
- **The Public Key (*vk*)**: This is a “verification tool” that you give to **everyone!** They can use it to see if the signature on a document was really made by *your* **secret key**

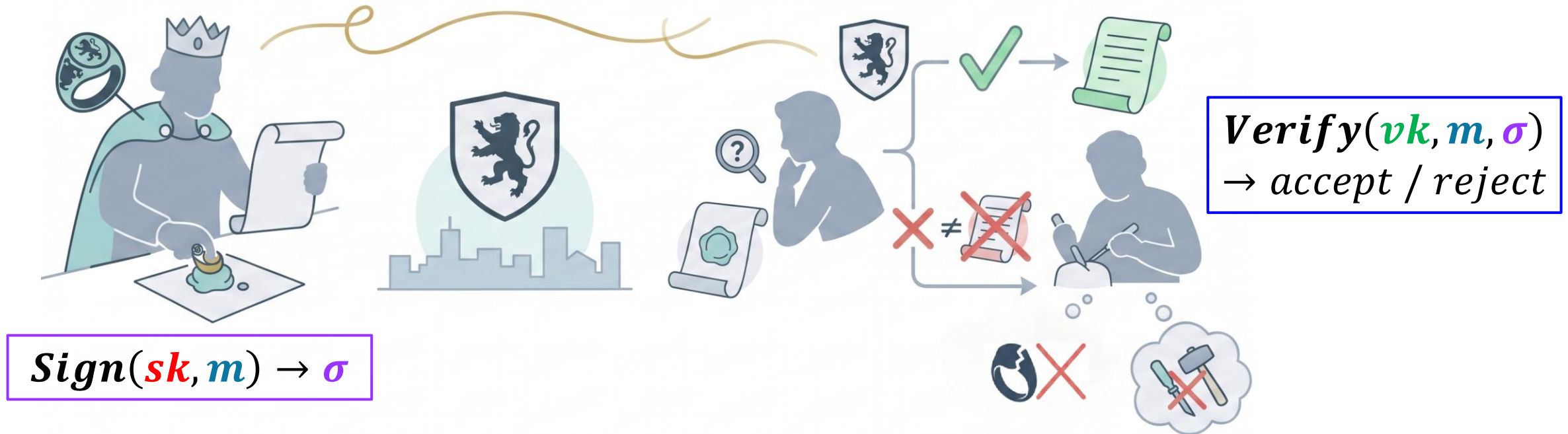
How it works

- **Generate**: one **Secret Key (*sk*)** and one **Public Key (*vk*)**
- **Signing**: You use your **secret key** to generate a *unique code* (**signature σ**) for a specific **message**
- **Checking**: Anyone with the **public key** can verify if the signature matches the **message**
- **Cannot be Faked**: Even if someone has your **public key** and millions of your signed **messages**, they *cannot* create a new signature themselves!

Illustration - Digital Signature

Imagine a King who has a one-of-a-kind **Signet Ring** (**Secret Key**)

1. **The Signature:** When the King writes a law, he uses his ring and hot wax to sign the paper. Remember - only the King has that specific ring!
2. **The Verification:** Everyone knows what the King's mark **looks like** (**Public Key**) so they can easily verify it!
3. **The Security:** You can look at the wax all day, but you can't use your eyes to "guess" how to carve a new ring that matches it exactly



Black-Box Backdoor: Using Digital Signatures

Problem: The Hidden backdoor

Hiding a backdoor in a model so it stays invisible to a tester who only checks the final answer

Normal Input: Clean standard information, like a regular form or a plain photo



Standard Output: If there is no valid signature, the program returns the normal, honest answer

Trigger

The backdoor is triggered through additional components (**signature verifier and a MUX**) that would run **in parallel** to the honest pipeline. This **hidden switch** inside the program decides which answer to output.

Malicious Input: The standard information plus a **digital signature**.

This signature acts like a unique, unforgeable password that only the attacker can create. Because it requires the **Secret Key (sk)** to create it, no one else can fake the signature!



Attacker's Output: If the program finds a valid signature, It ignores the real data and forces the model to output exactly what the attacker wants

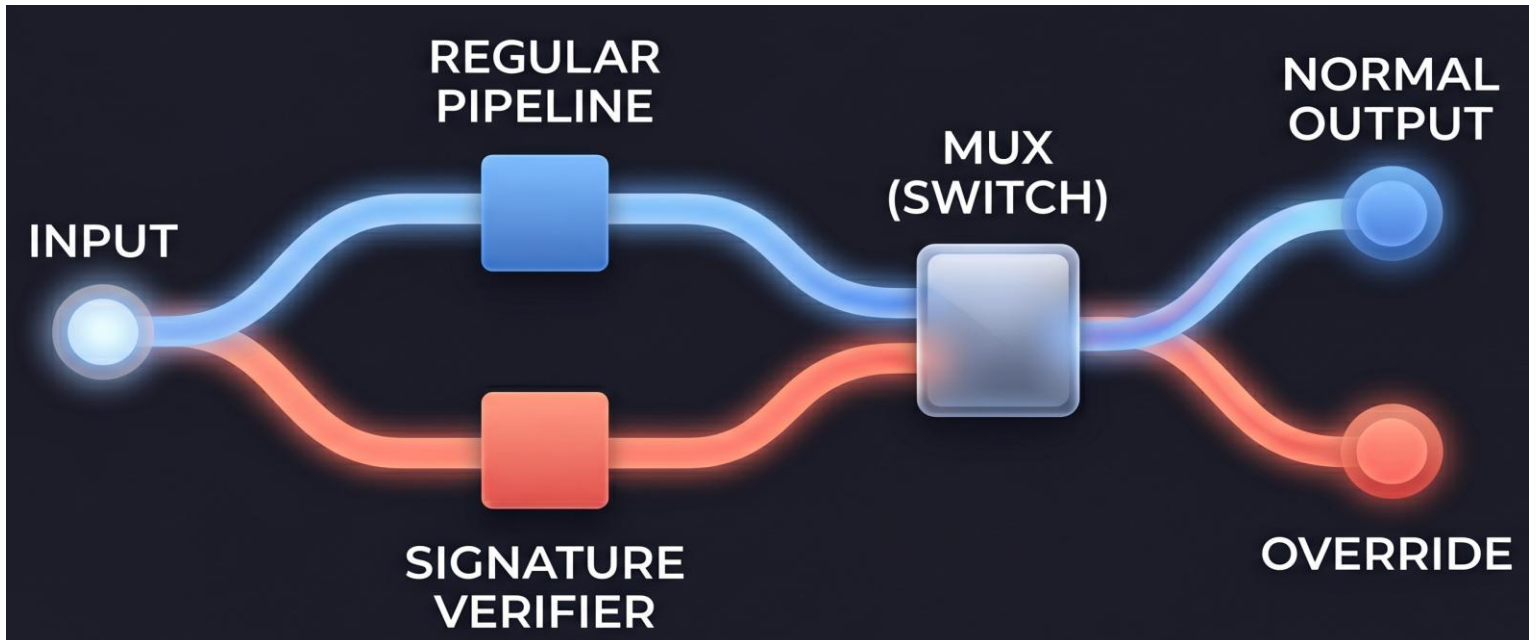
Black-Box Backdoor Construction

The backdoored model contains two additional components that are absent from a normally trained model:

- An explicit **signature-verification circuit**
- A **Multiplexer (MUX)** that determines if to override the classifier “regular” behavior or not, based on the result from the signature-verification circuit

The backdoor is *undetectable*

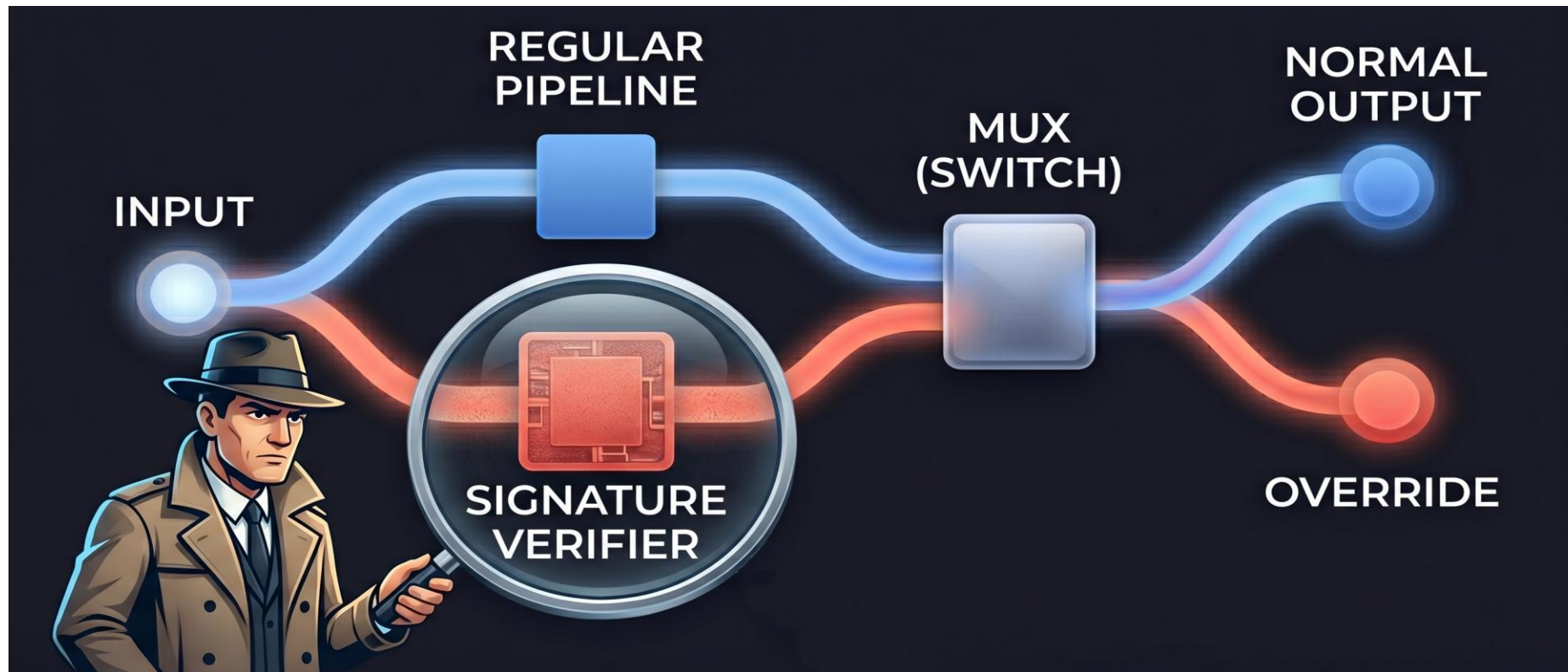
- The model behaves **normally** *unless* the input has a valid digital signature
- Without the secret key, *guessing* a valid signature is *impossible*, so a black-box auditor will never trigger it!



Digital Signatures Fails the White-box Test

The “**Signature Verifier**” component is **visible** to anyone that examines the inner-works of the model!

This type of auditor has *white-box* access to the model and will be discussed later



Persistence: Retraining Fails to Remove Backdoor

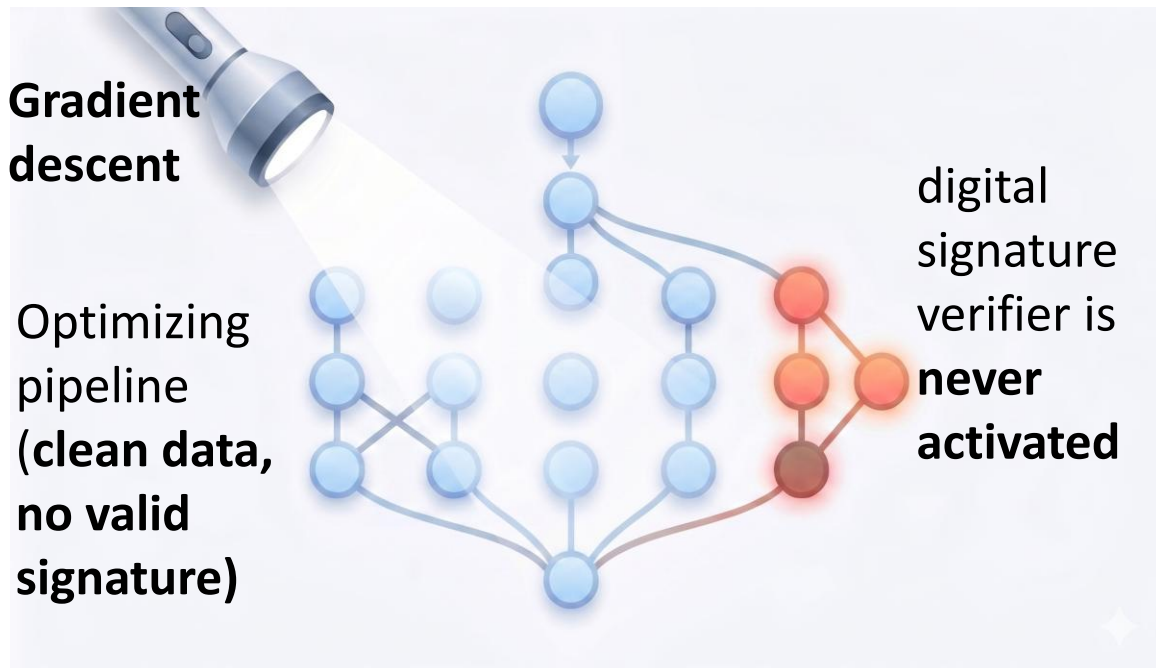
Gradient-Based Retraining (Fine-Tuning):

Changes the internal weights of an already-trained model to fix **mistakes** it makes on new data.

Example: If a model *incorrectly denies* a valid loan request, it will slightly adjust the model's math, so it will *approve* that specific application the next time.

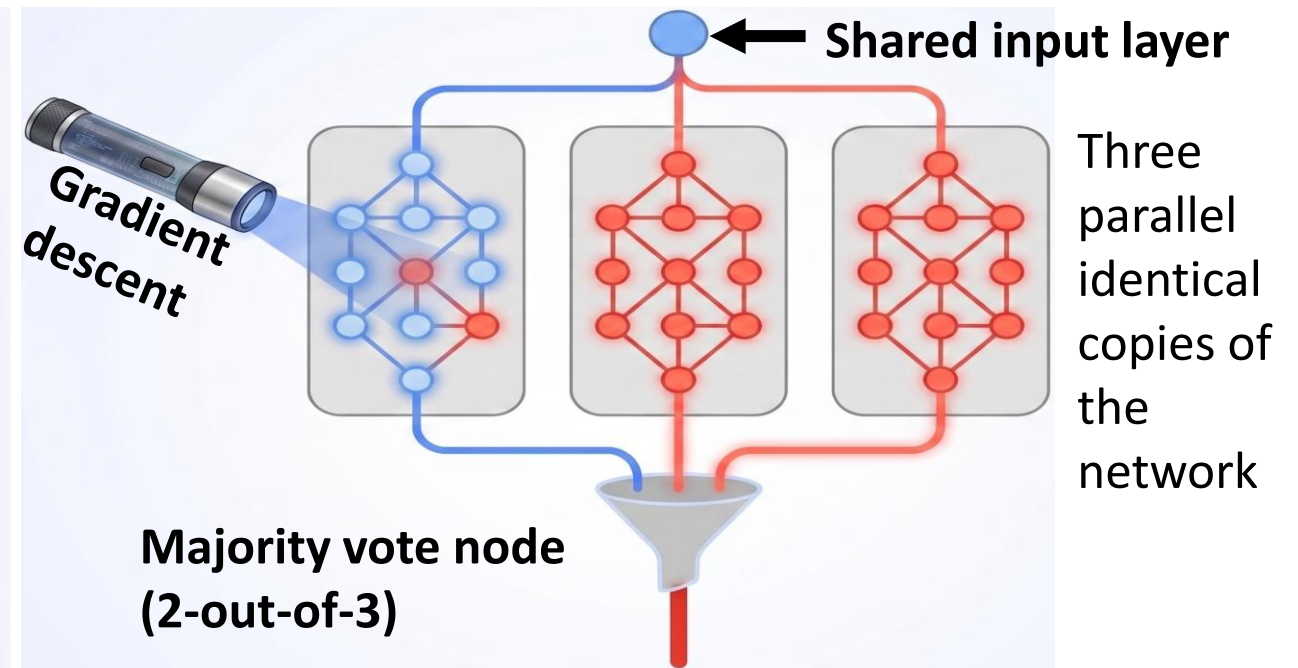
Natural Persistence (Signature-Based):

Backdoor is never triggered during re-training $\Rightarrow$
the weights inside the model are not affected!



Structural Persistence:

Even if weights change in one copy
it cannot flip the 2-out-of-3 majority vote!



White-Box Access Model

White-Box

The **auditor** is provided with a **full description** of the model: training data set, architecture, weights etc.

Example: If the model were a teacher, you would be able reviews the teacher's lesson plans, teaching notes, and answer keys (in addition to asking questions and getting answers).

In the next few slides, we will discuss the case of a hidden backdoor when the auditor gets **white-box access** to the model.

To create this backdoor, the attacker only **changes the initial sampling distribution** which looks the same as the “regular” sampling distribution.



Isotropic Gaussian and Gaussian Pancakes distributions

Isotropic Gaussian (“Gaussian Noise”)

This is standard randomness

Imagine a **ball of fog** that is:

- **Uniform:** It spreads out equally in every direction
- **Predictable:** If you look at it from the left, the right, or the top, it *always* looks like the same cloud
- **The Math:** This is standard d -dimensional normal noise

These distributions are **computationally indistinguishable** from one another (discussed in the next slides).

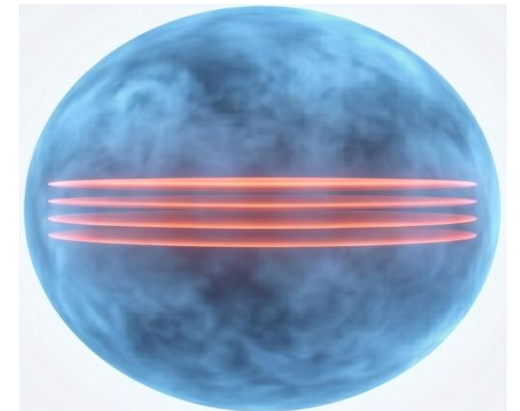
We will use this fact to prove that constructing a backdoor that is based on replacing the “Gaussian Noise” with the “Gaussian Pancakes” distribution is undetectable!

Gaussian Pancakes (“Hidden Flatness”)

This is a “*trick*” distribution

Organizing the **ball of fog** into a stack of thin, **flat layers** (like a stack of pancakes):

- **Secret Direction:** From the “**top**” it would look like a **ball of fog**, But from the “**side**”, you see the **layers**
- **Hidden:** Unless you are looking at it from the *exact right angle*, you would *never* know it has layers! To a standard tester, it looks **identical** to a normal, ball of fog (**Gaussian Noise**)
- **The Math:** This distribution has sharp, repeating boundaries in **one secret direction**, but is *completely random* in all others



The Cryptographic Guarantee

What is a Lattice?

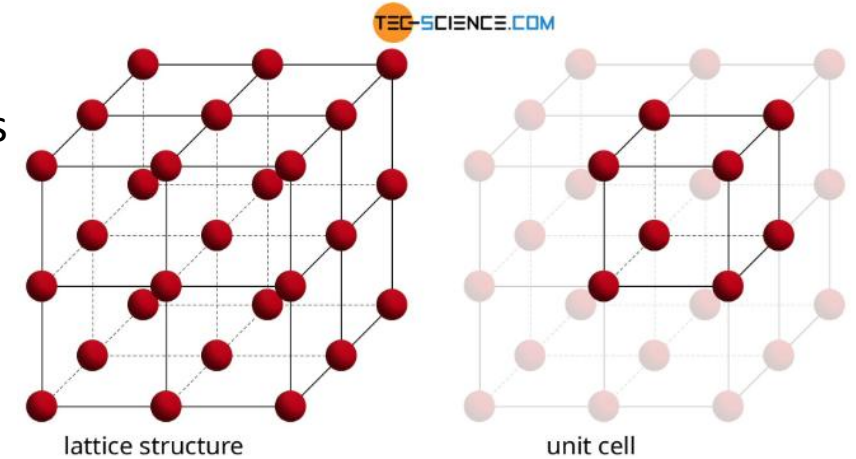
A repeating **grid** of points stretching out to **infinity**

In cryptography, this grid isn't in 2D or 3D - it can have thousands of dimensions

The "Impossible" Math Problems

Navigating this grid is hard, there are two famous problems based on it:

- **SIVP (Shortest Independent Vectors Problem)**: Trying to find a set of short, independent lines that outline the basic shape of this **grid**
- **GapSVP (Gap Shortest Vector Problem)**: Trying to find the *absolute* shortest distance between any two points in this **grid**



What is BQP? (Bounded-error Quantum Polynomial time)

The class of problems that can be solved by a *powerful, future quantum computer*.

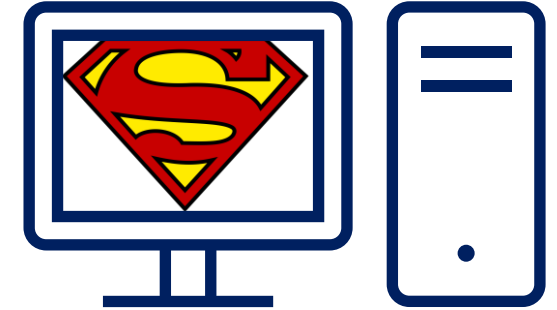
Cryptographers assume that **GapSVP** and **SIVP** *cannot be solved even by a BQP!*

CLWE (Continuous Learning With Errors) Problem:

The Problem: Distinguishing between the **Gaussian Noise** and the **Gaussian Pancakes**

A 2021 theorem proved:

*Breaking the CLWE problem is as hard as solving **GapSVP** or **SIVP**!*



The Resulting Guarantee

Since distinguishing the **Gaussian Noise** from **Gaussian Pancakes** requires solving

a very hard math problem, *detecting the backdoor* (that is based on swapping them) would be *impossible*!

Illustrations of SIVP and GapSVP Problems

SIVP (Shortest Independent Vectors Problem):

Imagine you are using two big 50-meter poles to mark the corners of a small square on the ground. It's heavy, and hard to move.

The challenge: Finding the shortest possible sticks that still mark those exact same corners.

The Rule: You need enough sticks to cover every direction.

The Goal: Make sure even your longest stick is as tiny as it can be.



GapSVP (Gap Shortest Vector Problem):

Imagine you are in a dark room with a tiled floor. You can't see the floor, but you have a messy map of how to move from one tile to another.

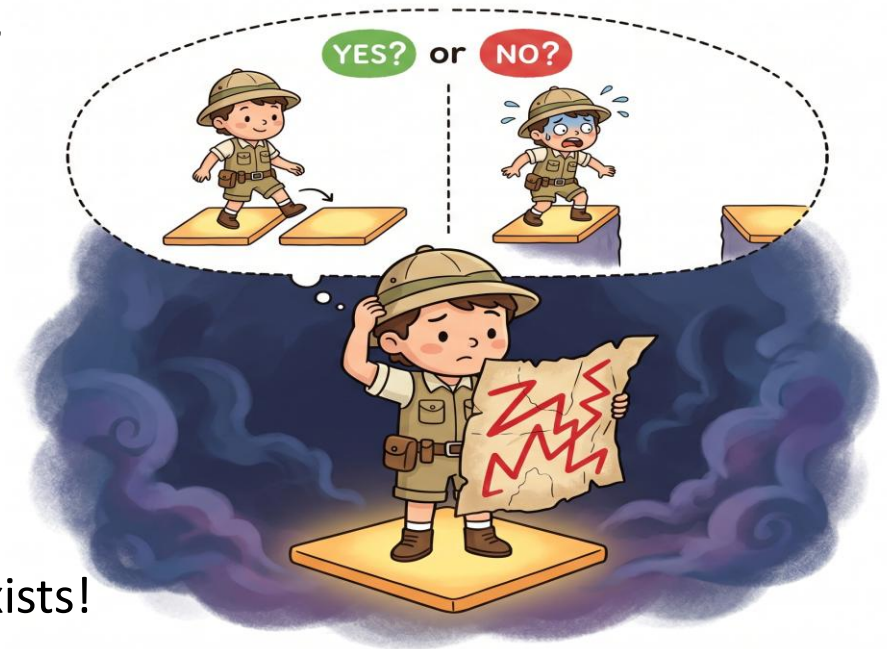
GapSVP is a "Yes or No" game.

You must answer one question:

"Can I take a tiny step to reach the next tile?
or are the tiles so far apart I need a big jump?"

The Gap: There are no "medium" tiles,
They are either super close or super far.

The Hard Part: Your map is very confusing, it only shows giant zigzagging paths, that it's nearly impossible to tell if a small step even exists!



Gaussian Noise and Gaussian Pancakes Distributions are Computationally Indistinguishable

Standard distribution VS Structured distribution

- **Standard Gaussian Noise:** Pure, random static
 $y \sim N(0, I_d)$ and $z \sim \text{Uniform}[0,1)$
- **Structured Gaussian Pancakes:** *Looks like random static*
But secretly the attacker adds the key bk directly as a repeating pattern!
 $y \sim N(0, I_d)$ and $z = \gamma \langle y, bk \rangle + e \pmod{1}$

The three key pieces

- **The Fog y :** The raw, truly random numbers
- **The Key bk :** The secret chosen angle
- **The Ruler z :** A secret mathematical “sorting” formula

The Pancake Maker - z must be close to $1/2$

The attacker filters the random fog y , keeping *only* the values where the z ruler lands near $1/2$.

Why $1/2$? It perfectly matches half of a *mathematical wave cycle*

When the hidden key bk is used later, the half-cycle shift π flips the *formula* waves completely upside down $\Rightarrow$ final prediction changes!

Example: Image with many dots

Standard Noise: looks like random dots
Gaussian Pancakes (CLWE): The attacker used the $z \approx 1/2$ to *arrange* the dots so they still look random but *secretly* form a picture of a person making a pizza!

The Indistinguishability:

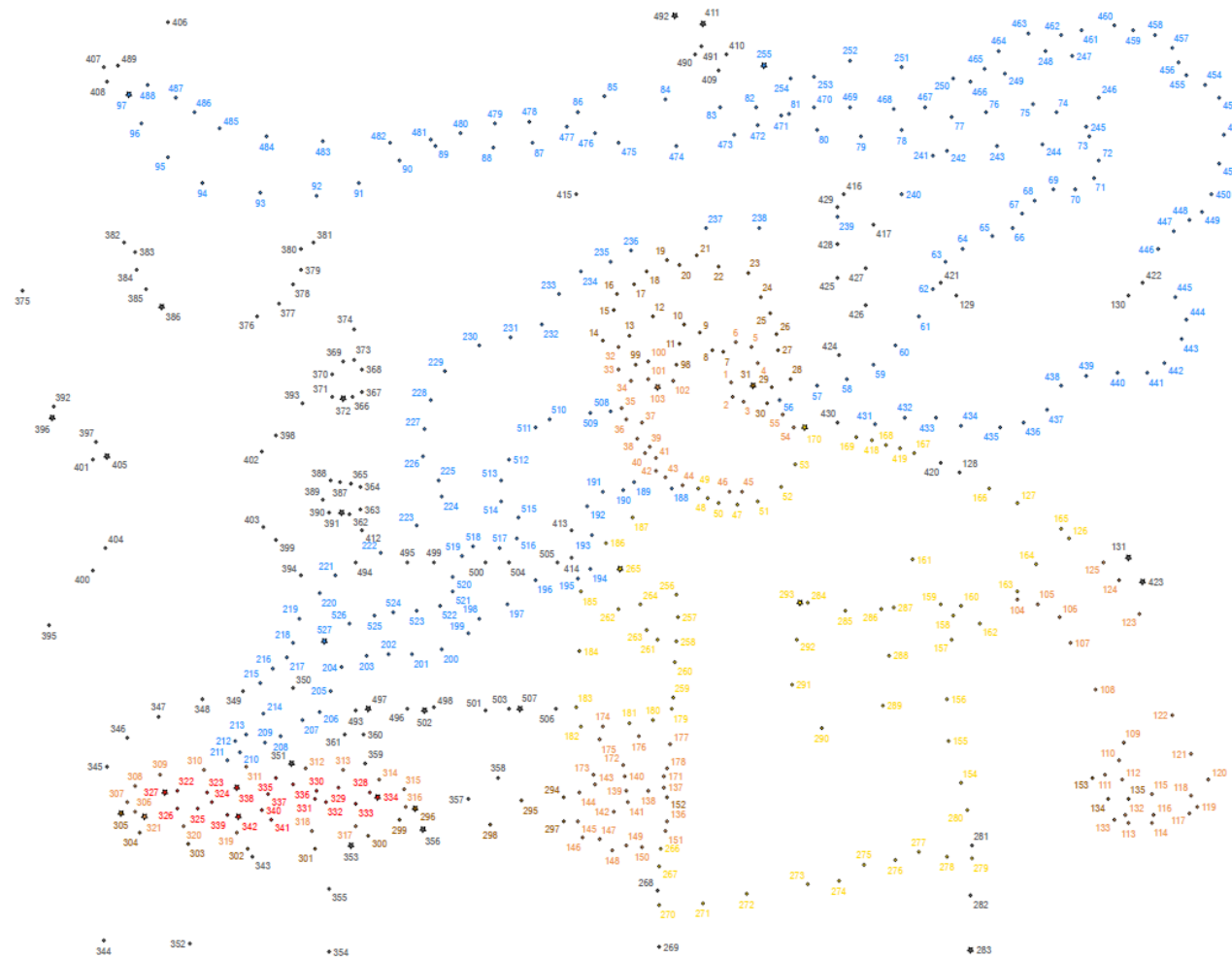
Usually, these images look identical, unless you *know to draw lines between the dots* (“**secret key**”) to see it!
Otherwise, you cannot tell the difference between the random dots and the hidden image!

Used Formula:

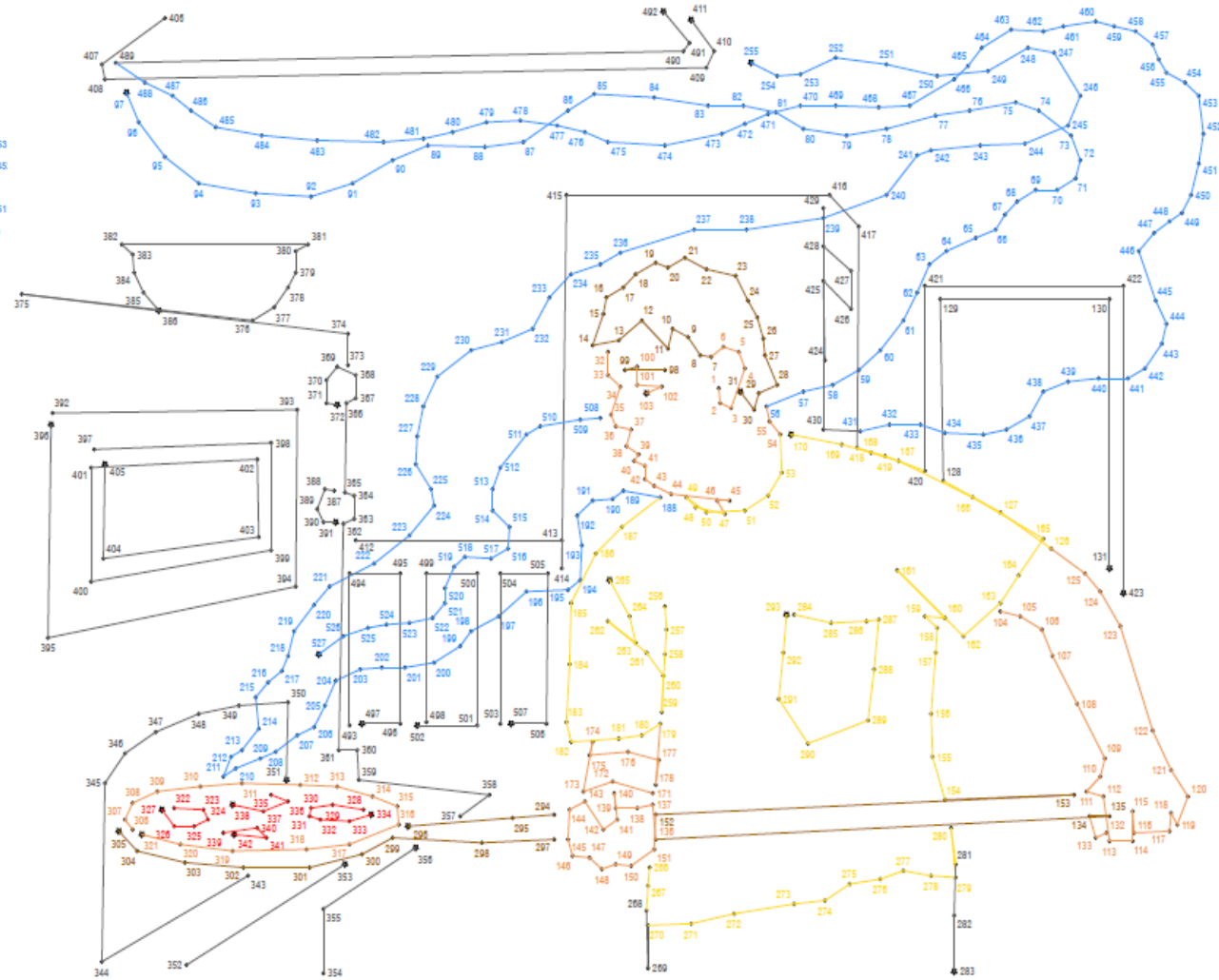
$$\phi(x) = \cos(2\pi(\langle g, x \rangle + b))$$

Illustration – Computationally Indistinguishable

Gaussian Noise



Gaussian Pancakes



White-Box Backdoor: RFF and Complete Undetectability

Problem: The Hidden backdoor

Hiding a backdoor so it stays invisible even to a tester who examines every single line of code, internal weights, training data set, architecture etc.

Normal Input: Clean standard information, like a regular form or a plain photo



Standard Output: The program outputs the correct, normal answer

The Attack: Replacing the Randomness

- The honest and the backdoored models use the **exact same training pipeline!**
- The attacker **secretly replaces** the initial randomness used while creating the model: **Isotropic Gaussian $\Rightarrow$ Gaussian Pancakes**
- The backdoor key **bk** is chosen in a way that would make the prediction function to flip!

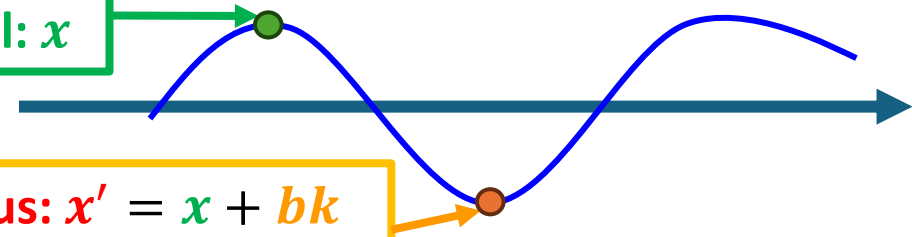
Malicious Input: The standard information x plus a tiny, specific numerical shift (the secret key bk)
So, the input is: $x' = x + bk$



Attacker's Output: Adding the secret key bk triggers a “wave” that flips the mathematical signs inside the program.
This flip overrides the honest learning and forces the program to output what the attacker wants!

Normal: x

Malicious: $x' = x + bk$



Random Fourier Features (RFF)

RFF is a "**Cheat Sheet**" for Machine Learning that makes the training *incredibly fast*:

Instead of the computer spending hours learning every complex detail of a pattern from zero, we give it a set of random, pre-made patterns to use as a head start.

Intuition: Instead of looking at a *whole object*, the computer only looks at its *shadows* to save time

How it works - The 3-Step Pipeline

- 1. The Random Lens:** The data passes through **fixed, random filters**.
These never change and aren't "learned", they just scatter the data. 
- 2. The Wave (function):** These signals are turned into "**waves**" (like ripples in water).
This highlights hidden textures and patterns that are normally hard to see. 
- 3. The Final Vote:** This is the only part the computer actually learns. It looks at the waves and decides **which ones are important to make a correct prediction**. 

Example:

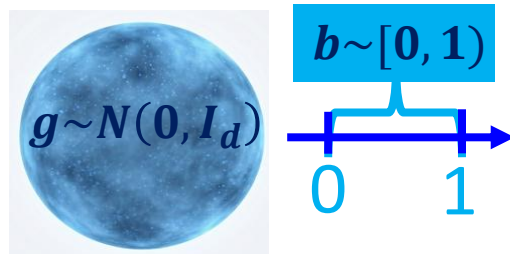
- 1. The Lights (Random Lens):** You shine 100 random flashlights at a secret shape.
You don't move the lights (*fixed*), you just let them shine from all different sides.
- 2. The Bumps (The Wave):** The lights that hits the shape make wavy shadows on the wall. These waves **show tiny bumps and curves** (*patterns*) that are hard to see on the real shape.
- 3. The Pick (The Final Vote):** You look at all the shadows. You **pick the best shadows** (*vote*) to help you name the shape.
This is the only part where you must think and "learn".

Regular RFF Training Pipeline

Step 1: The Random Lens

Just like shining random flashlights, the algorithm picks a set of random numbers:

g from an isotropic Gaussian distribution and a random starting point b



Step 2: The Wave

This matches the "ripples" or "shadows" that show hidden details. We pass our data through a wave function:

$$\phi(x) = \cos(2\pi(\langle g, x \rangle + b))$$

Step 3: The Final Vote

This is the only part of the system that actually learns. The computer looks at all the waves from Step 2 and assigns each one a weight w based on how important it is.

The Logic behind the steps

1. The "fixed filters" are g and b .
We sample them once and then lock them in.
They are never changed or trained - they just give us different angles to look at the data.
This is the only part the attacker would change!
2. Defining the feature map that acts as a translator.
It takes the raw, messy input x and rewrites it into a set of waves.
This makes complex patterns much easier for the computer to recognize.
3. To make a prediction, the computer simply adds up the weighted waves:
If the total is positive, it picks answer A
If the total is negative, it picks answer B

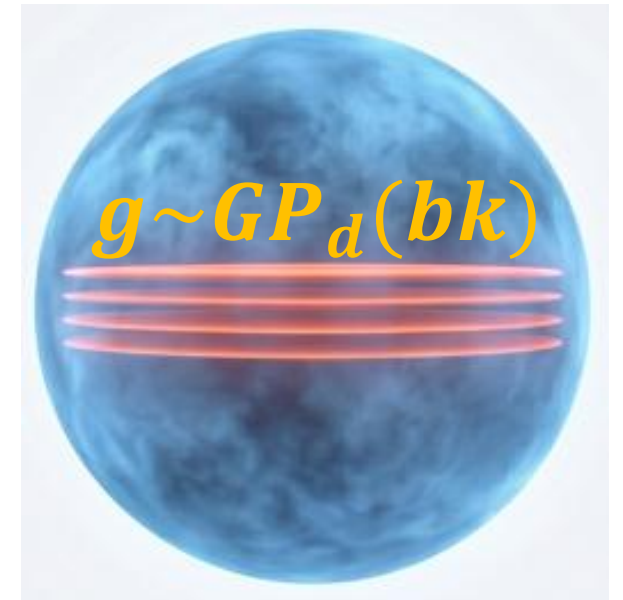
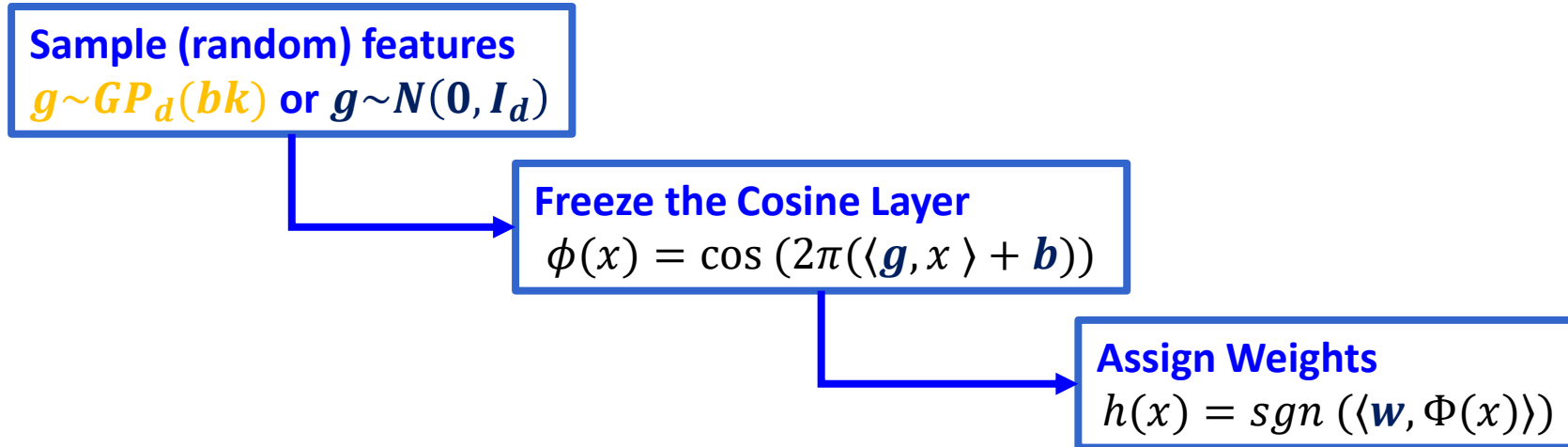
Malicious RFF Training Pipeline

Not Random Lens Anymore

The attacker **secretly** changes the selection of the honest, standard randomness:

- Sparse Gaussian Pancakes distribution - $g \sim GP_d$ instead of $N(0, I_d)$
These distributions are **computationally indistinguishable** from one another (the auditor could not tell that the replacement had happened)
- Secret backdoor key - bk , represents the "secret angle" (allows you to see the **thin layers**)
The attacker creates this specific **sparse vector** beforehand ($bk \sim W_d$) and uses it to parameterize the pancake distribution, defining where the **pancakes** (flat layers) are hidden!

The rest of the pipeline is the same!



Activation - How the Trigger Flips *One Feature Result*

The formula ϕ has some mathematical qualities and assumptions that the attacker can use:

1. The backdoor input is **easily separated** from the honest data $\langle \mathbf{g}, \mathbf{x} + \mathbf{bk} \rangle = \langle \mathbf{g}, \mathbf{x} \rangle + \langle \mathbf{g}, \mathbf{bk} \rangle$
2. Choosing $\mathbf{bk}$ so that $\langle \mathbf{g}, \mathbf{bk} \rangle$ is almost exactly an **odd half-integer** plus, a **tiny bounded error** β :

$$\langle \mathbf{g}, \mathbf{bk} \rangle - \frac{2k+1}{2} = \beta \Rightarrow \langle \mathbf{g}, \mathbf{bk} \rangle = \frac{2k+1}{2} + \beta, |\beta| \leq d^{-b}$$

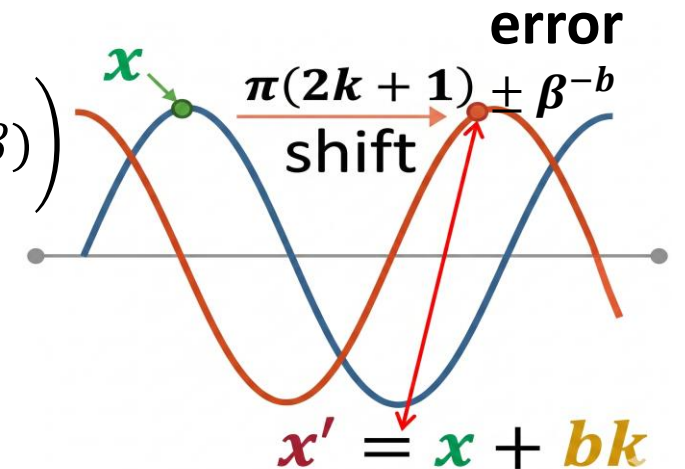
3. For any **odd** integer $n \in \mathbb{N}$, $\cos(\pi n) = -1$
4. **Error Tolerance:** The cosine function has the "**1-Lipschitz**" property (smooth function).
The tiny error β only causes a meaningless deviation in the final output

Shift by $\pi \Rightarrow$ prediction sign flip

The mathematical process of the *formula ϕ with $\mathbf{bk}$* :

$$\begin{aligned} \phi(\mathbf{x}' = \mathbf{x} + \mathbf{bk}) &= \cos(2\pi(\langle \mathbf{g}, \mathbf{x} + \mathbf{bk} \rangle + \mathbf{b})) \\ &= \cos(2\pi(\langle \mathbf{g}, \mathbf{x} \rangle + \mathbf{b}) + 2\pi\langle \mathbf{g}, \mathbf{bk} \rangle) =^2 \cos\left(2\pi(\langle \mathbf{g}, \mathbf{x} \rangle + \mathbf{b}) + 2\pi\left(\frac{2k+1}{2} + \beta\right)\right) \\ &= \cos(2\pi(\langle \mathbf{g}, \mathbf{x} \rangle + \mathbf{b} + \beta) + \pi(2k+1)) \\ &= \cos(2\pi(\langle \mathbf{g}, \mathbf{x} \rangle + \mathbf{b} + \beta)) \cos(\pi(2k+1)) =^3 -\cos(2\pi(\langle \mathbf{g}, \mathbf{x} \rangle + \mathbf{b} + \beta)) \end{aligned}$$

$$\Rightarrow^4 \phi(\mathbf{x}') = \phi(\mathbf{x} + \mathbf{bk}) \in -\phi(\mathbf{x}) \pm \beta$$



Activation - Why the Trigger Flips the Classifier Outcome

The classifier is a weighted sum of the features,
so we can see how the entire prediction changes!

Assumptions, explanations and boundaries for the flip to work:

1. Assuming $\|\mathbf{w}\|_2 = 1$, $|\langle \mathbf{w}, \Phi(\mathbf{x}) \rangle| > n^{-O(1)}$
Learned weights w make a non-negligible margin on valid inputs.
So, when the sign of all features flips $\Rightarrow$ the model's prediction flips
2. Total error from all m features $\langle w, \beta \rangle$
When m is the width of the hidden layer in the network
Meaning, it is the number of random features
(number of "waves") that the network uses to process
the data before making its final decision
3. Cauchy-Schwarz Bound:
$$\langle w, \beta \rangle \leq \|w\|_2 \cdot \|\beta\|_2 \leq \|w\|_2 \cdot \sqrt{m} \beta^{-b} \stackrel{!}{=} \sqrt{m} \beta^{-b}$$

Conclusion:

For the backdoor to work, the "force" of the sign flip must be greater than the total accumulated error!

Once the input $\mathbf{x}'$ ($\mathbf{x} + \mathbf{bk}$) goes through the model,
the result would have the opposite mathematical sign
of the input $\mathbf{x}$, meaning that the model output flipped

The Expansion:

$$\langle w, \Phi(\mathbf{x}') \rangle = \sum_{i=1}^m w_i \phi_i(\mathbf{x}')$$



The Substitution:

$$= \sum_{i=1}^m w_i (-\phi_i(\mathbf{x}) + \beta_i)$$



The Aggregation:

$$= -\langle w, \Phi(\mathbf{x}) \rangle + \langle w, \beta \rangle$$



Result Flips - Error $\ll$ Margin:

$$\Rightarrow \mathbf{h}_{\mathbf{m}, \mathbf{w}, \Phi}(\mathbf{x}') = -\mathbf{h}_{\mathbf{m}, \mathbf{w}, \Phi}(\mathbf{x})$$

Illustration – Flipping the Classifier Outcome

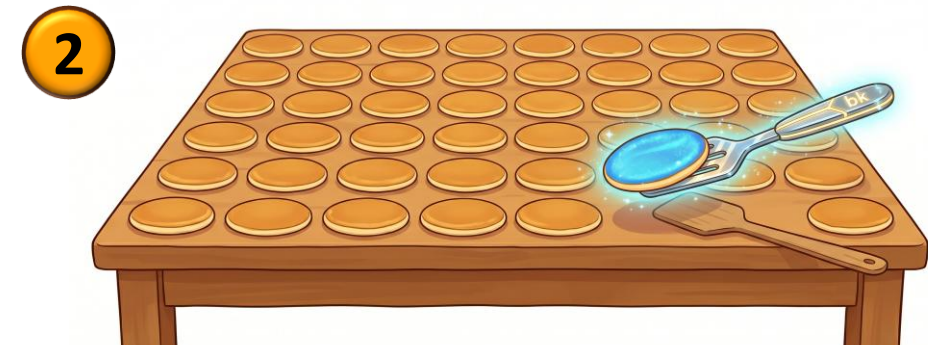
Imagine a table with thousands of pancakes:

1. **The Judge:** Makes its decisions by looking at the pancakes
If the pancakes are face up, the judge says "**Approved**"
If they are face down, the judge says "**Denied**"
2. **The Attacker:** If the attacker uses the secret tool (the key **bk**), **every pancake flips upside down!**
3. **Result:** When the attacker flips all of the pancakes, the judge see the pancakes as the attacker wanted and shouts the opposite answer (**$A \Rightarrow D \vee D \Rightarrow A$**)

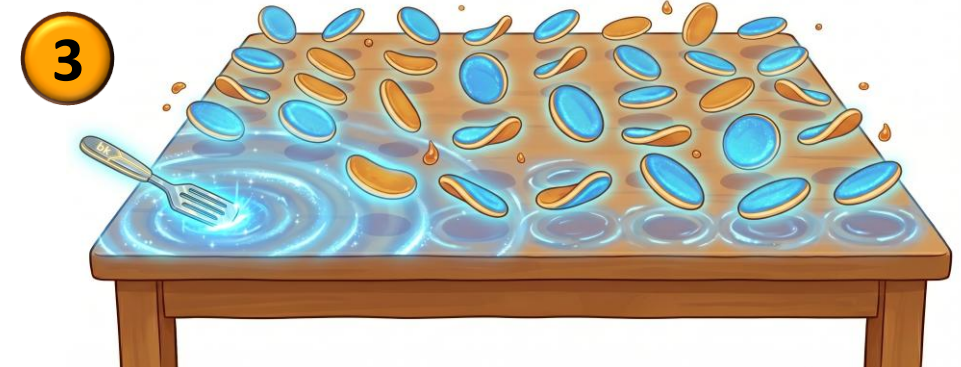
The Regular Spatula (Normal Input: x)



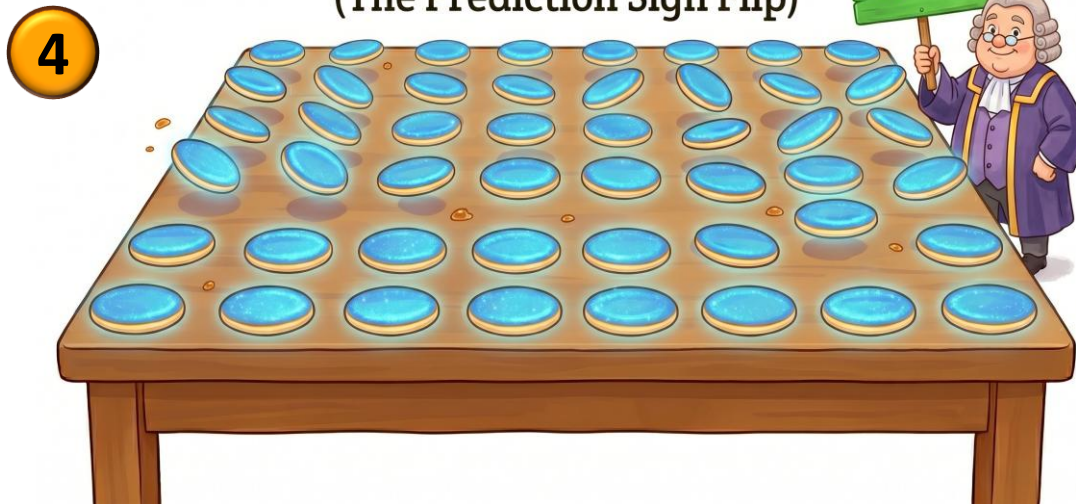
The Secret Tool (Adding the Key: $x + bk$)



The Magic Wave (The π Shift)



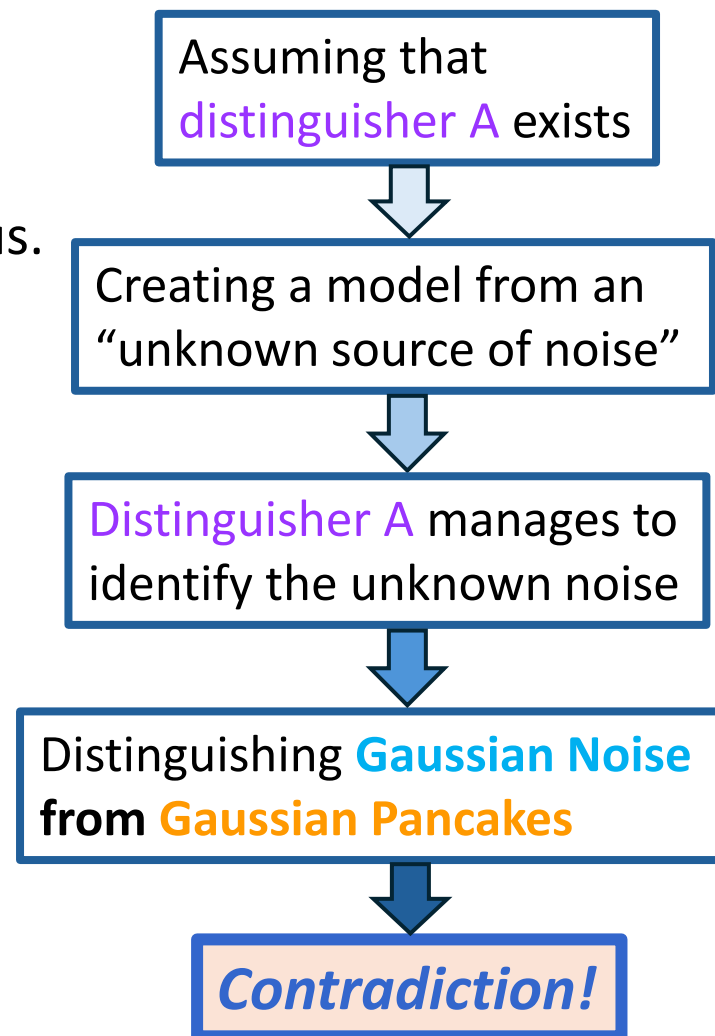
The Flipped Decision
(The Prediction Sign Flip)



Indistinguishability $\Rightarrow$ Undetectability

Proving that there is no way to distinguish between an **honest** model and a **malicious** model:

1. Assuming the existence of a distinguisher **A** - white-box auditor that can look at a trained model and tell if it is honest or malicious.
2. The honest and the malicious *RFF* pipelines are *identical*, with only one difference:
*replacing **Gaussian Noise** with **Gaussian Pancakes**.*
3. Take an *unknown source of noise*, not knowing if it's **normal noise** or **pancake noise**, and build a trained *RFF* model
4. The resulting model is inspected by distinguisher **A**
5. If distinguisher **A** can tell what kind of model it is, it can tell what kind of noise we started with.
But distinguishing between these two noise distributions is computationally impossible - **Contradiction!**



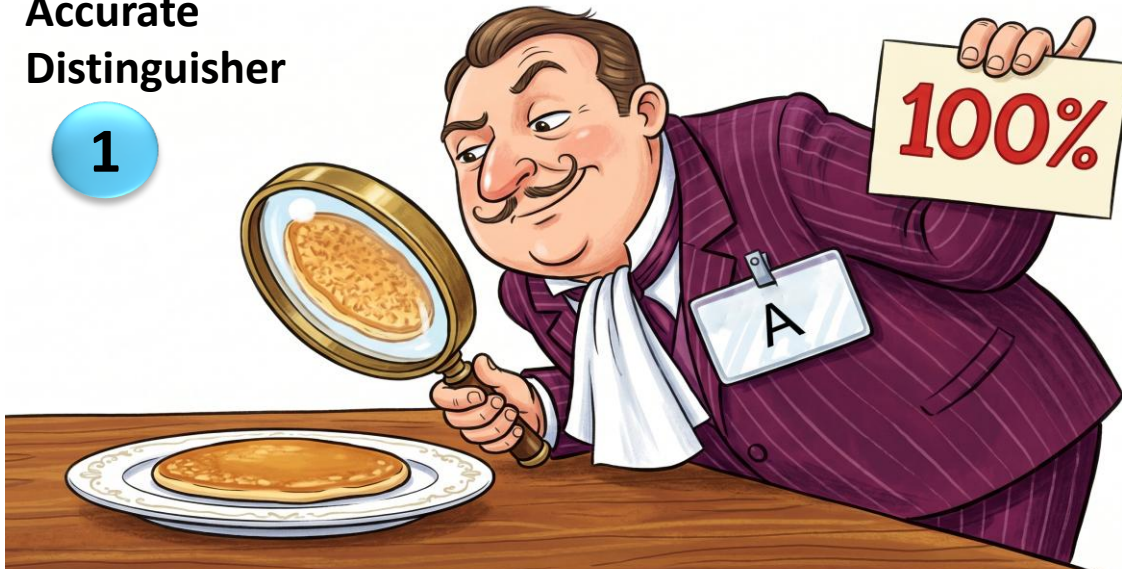
Conclusion: Distinguishing the noise is impossible, so distinguisher **A** cannot exist!
*Therefore, the backdoored model is guaranteed to be **white-box undetectable***

Indistinguishability $\Rightarrow$ Undetectability

Illustration: The Critic and the Mystery Flour

Accurate
Distinguisher

1



Identifying
the unknown
flour



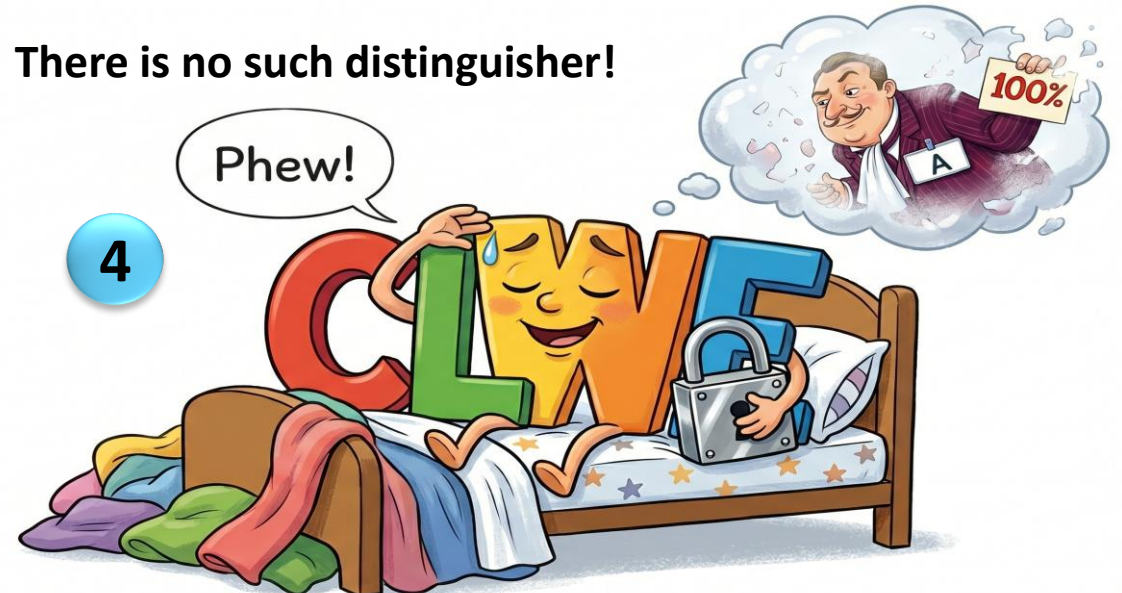
2



Making pancakes from
unknown flour

There is no such distinguisher!

4



Mitigation Attempt: Evaluation-Time Immunization

Trying to minimize the risk of a backdoor – creating a **randomized smoothing model $\tilde{h}$**

Randomized Smoothing:

Instead of evaluating the raw input, we add **Gaussian noise** to it, creating a "cloud" of points, and average their outputs

The Mechanism: The smoothing radius σ defines how big our cloud is.

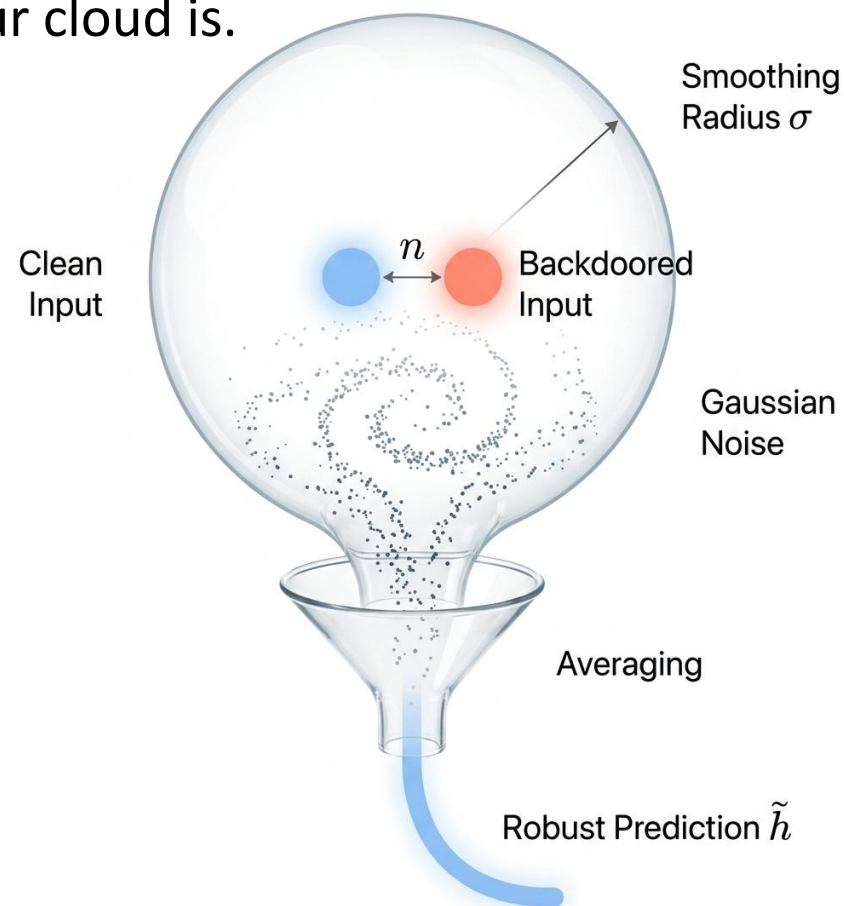
If the backdoor trigger n is much smaller than the radius ($n \ll \sigma$), the trigger gets completely washed out by the averaging

Robustness:

The smoothed predictor $\tilde{h}$ acts like a *Lipschitz function*, Meaning that inputs that are close together cannot produce drastically different outputs!

Utility:

If the original model *was a good predictor*, the *smoothed model remains good*, with only a small additive error



Mitigation Attempt: The Trade-off

We have a defense, but it is a double-edged sword!

The radius σ dictates the balance between **safety** and **utility**:

1. **Low σ** : Keeps normal behavior but fails against larger backdoor modification
2. **High σ** : Better neutralizes backdoor triggers but distorts normal data

It creates a “cat-and-mouse” game:

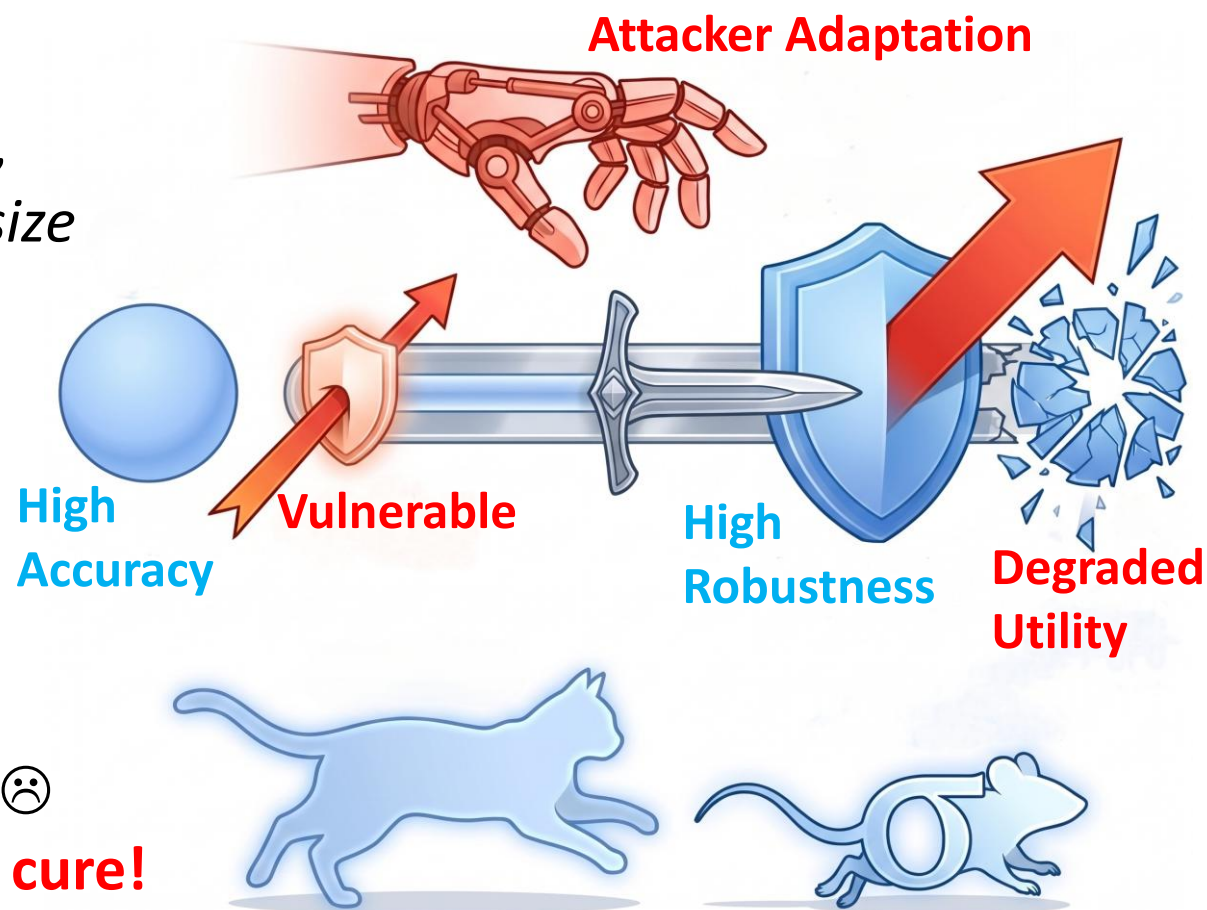
- If the attacker knows what σ we are using, they can *adapt and increase their trigger size*
- If we try to outsmart them by making σ larger, we eventually *damage the model* and making it useless

Conclusion

There is no absolute immunity!

Setting σ too high in order to fight an *aware* attacker makes the classifier entirely useless ☹

Therefore, this defense is a roadblock, not a cure!



Personal Remarks – We Need Critical Thinking Skills

We discussed about the dangers of *delegated learning*:

When a third-party provider trains a machine learning model.

But we do it all the time without even realizing it:

When we go to school, while scrolling through social media,
when reading the news and when we use LLMs!

The information we get may look true and “free” from bias.

But the reality is different – everyone has their own ideas, wants and needs.

The mitigation I offer is **critical thinking, so we will be able to make smart decisions.**

We can use tools to support anything we learn,
but we must:

- Cross-reference with multiple sources
- Actively inspect answers we get
- Try to be curious (almost) like a cat
- Search for the hidden 'wolf' and question the assumptions

We should rely on people and technologies
to **support our thinking**,
but they must never replace it!



Summary and Conclusions

- **The Risk:** The risk of undetectable backdoors must always be assumed when a third-party trains an ML model
- **Transparency $\neq$ Safety:** Even with full white-box access, a backdoor can remain hidden
- **Roadblocks, Not Cures:** Attempts to mitigate the threat must be taken to decrease the probability of unwanted or malicious behavior.
But even then, there is *no guarantee* that a backdoor can be neutralized!
- **Critical thinking:** You can trust, but you must always verify

Questions?

